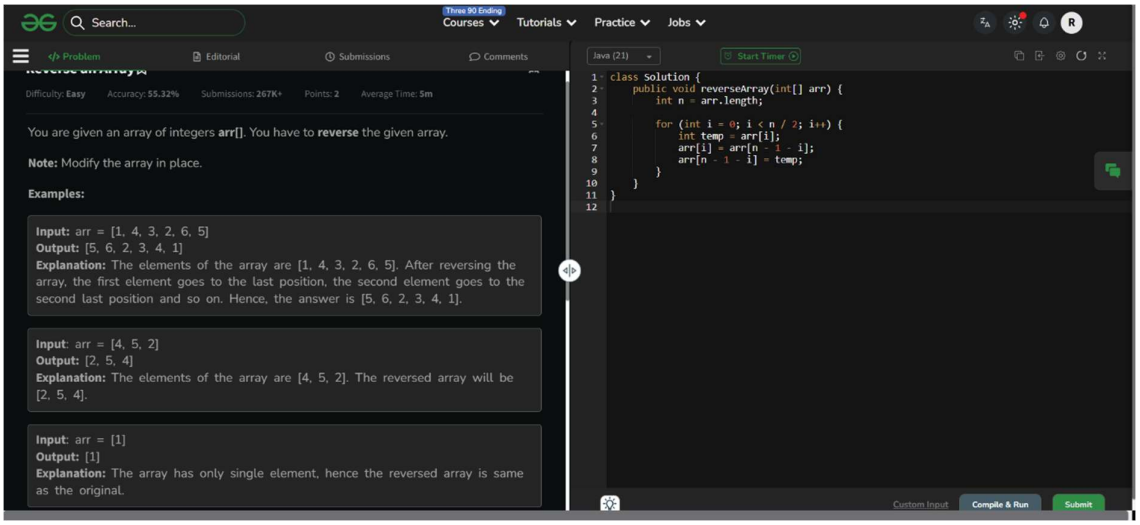
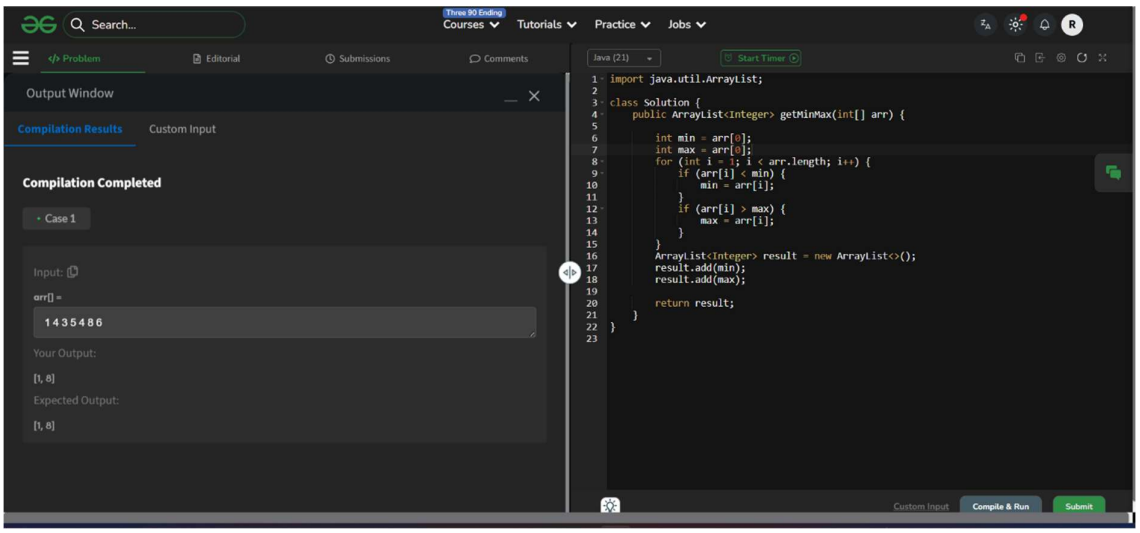
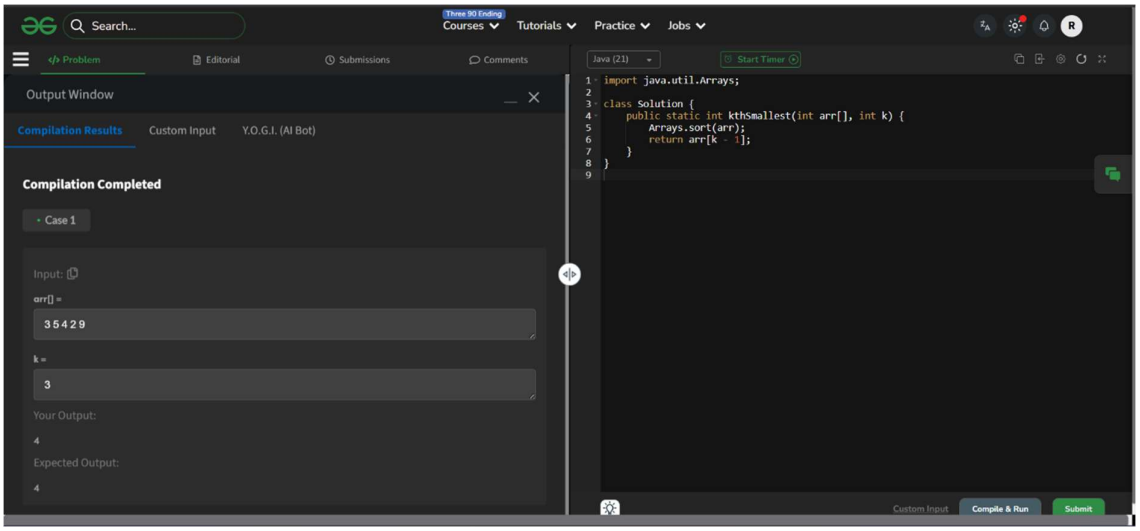
1.


Search...
Three 90 Ending
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Difficulty: Easy
Accuracy: 55.32%
Submissions: 267K+
Points: 2
Average Time: 5m
You are given an array of integers arr[]. You have to reverse the given array.
Note: Modify the array in place.
Examples:
Input: arr = [1, 4, 3, 2, 6, 5]
Output: [5, 6, 2, 3, 4, 1]
Explanation: The elements of the array are [1, 4, 3, 2, 6, 5]. After reversing the array, the first element goes to the last position, the second element goes to the second last position and so on. Hence, the answer is [5, 6, 2, 3, 4, 1].
Input: arr = [4, 5, 2]
Output: [2, 5, 4]
Explanation: The elements of the array are [4, 5, 2]. The reversed array will be [2, 5, 4].
Input: arr = [1]
Output: [1]
Explanation: The array has only single element, hence the reversed array is same as the original.
Java (21)
Start Timer
class Solution {
public void reverseArray(int[] arr) {
int n = arr.length;
for (int i = 0; i < n / 2; i++) {
int temp = arr[i];
arr[i] = arr[n - i - i];
arr[n - 1 - i] = temp;
}
}
}
Custom Input
Compile & Run
Submit

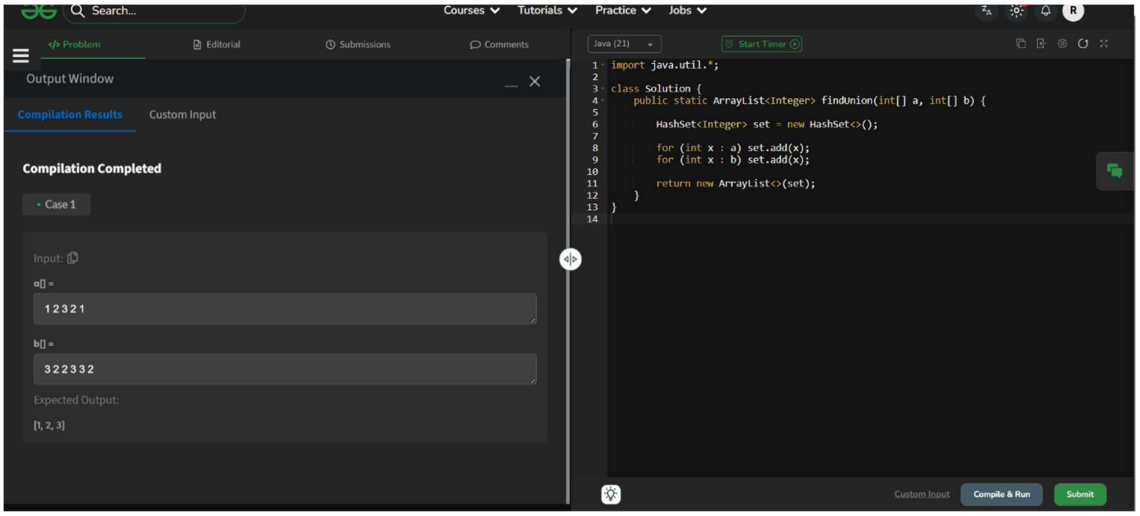
2.


Search...
Three 90 Ending
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Compilation Completed
Case 1
Input:
arr[] =
1 4 3 5 4 8 6
Your Output:
[1, 8]
Expected Output:
[1, 8]
Java (21)
Start Timer
import java.util.ArrayList;
class Solution {
public ArrayList<Integer> getMinMax(int[] arr) {
int min = arr[0];
int max = arr[0];
for (int i = 1; i < arr.length; i++) {
if (arr[i] < min) {
min = arr[i];
}
if (arr[i] > max) {
max = arr[i];
}
}
ArrayList<Integer> result = new ArrayList<>();
result.add(min);
result.add(max);
return result;
}
}
Custom Input
Compile & Run
Submit

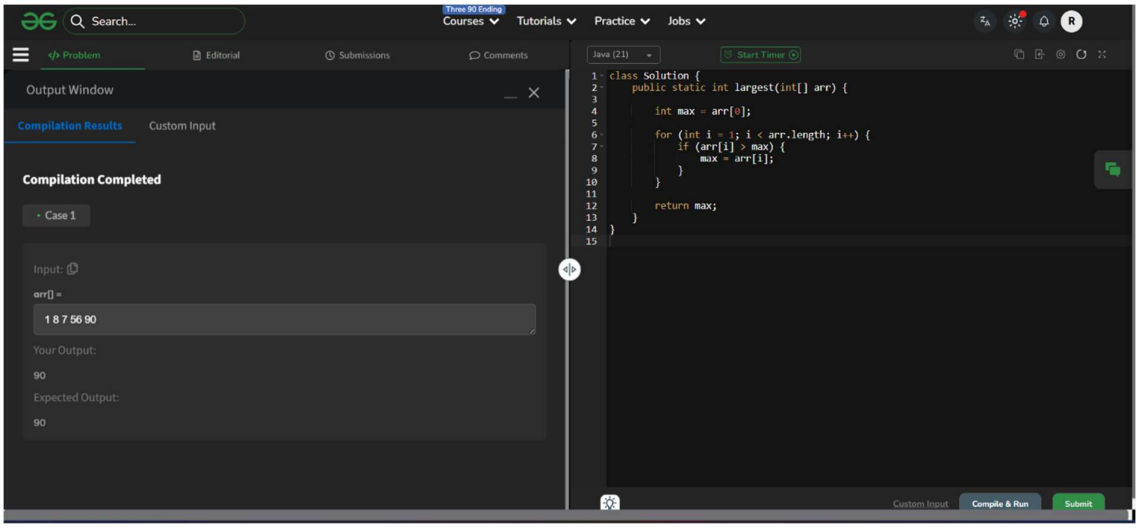
3.


Search...
Three 90 Ending
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Y.O.G.I. (AI Bot)
Compilation Completed
Case 1
Input:
arr[] =
3 5 4 2 9
k =
3
Your Output:
4
Expected Output:
4
Java (21)
Start Timer
import java.util.Arrays;
class Solution {
public static int kthSmallest(int arr[], int k) {
Arrays.sort(arr);
return arr[k - 1];
}
}
Custom Input
Compile & Run
Submit

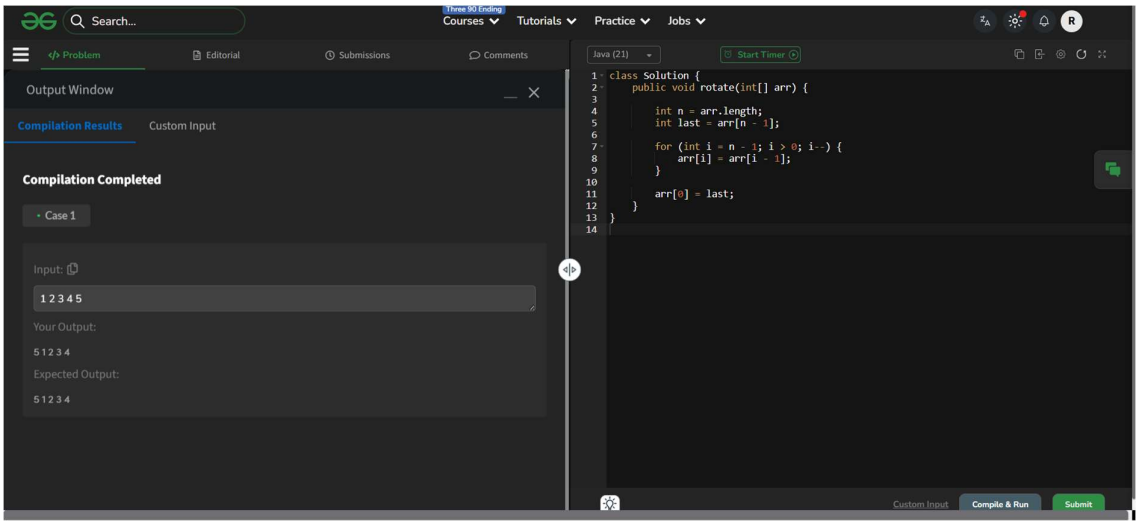
4.


Search...
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Compilation Completed
Case 1
Input:
a[] =
1 2 3 2 1
b[] =
3 2 2 3 3 2
Expected Output:
[1, 2, 3]
Java (21)
Start Timer
import java.util.*;
class Solution {
public static ArrayList<Integer> findUnion(int[] a, int[] b) {
HashSet<Integer> set = new HashSet<>();
for (int x : a) set.add(x);
for (int x : b) set.add(x);
return new ArrayList<>(set);
}
}
Custom Input
Compile & Run
Submit

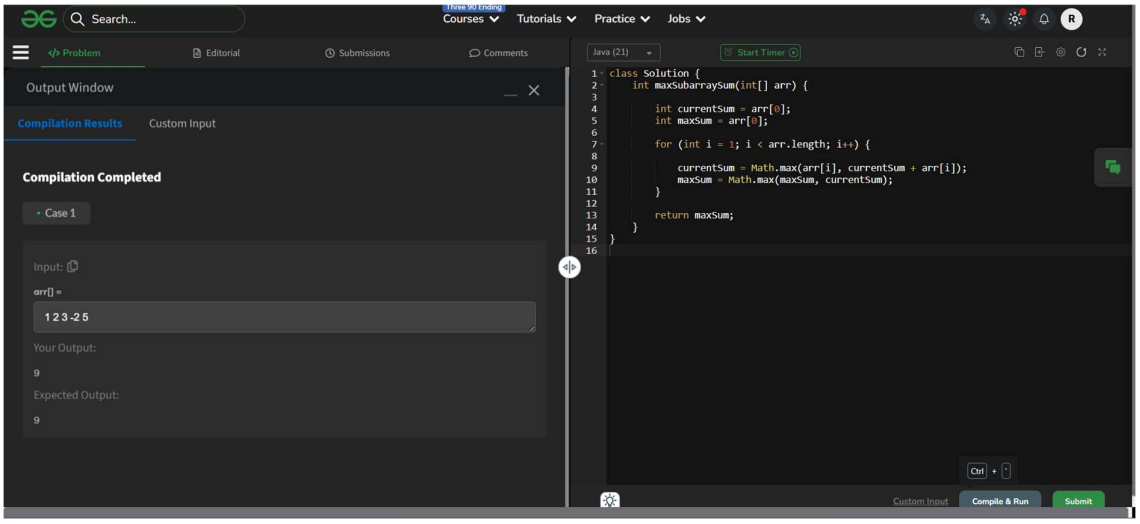
5.


Search...
Three 90 Ending
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Compilation Completed
Case 1
Input:
arr[] =
1 8 7 56 90
Your Output:
90
Expected Output:
90
Java (21)
Start Timer
class Solution {
public static int largest(int[] arr) {
int max = arr[0];
for (int i = 1; i < arr.length; i++) {
if (arr[i] > max) {
max = arr[i];
}
}
return max;
}
}
Custom Input
Compile & Run
Submit

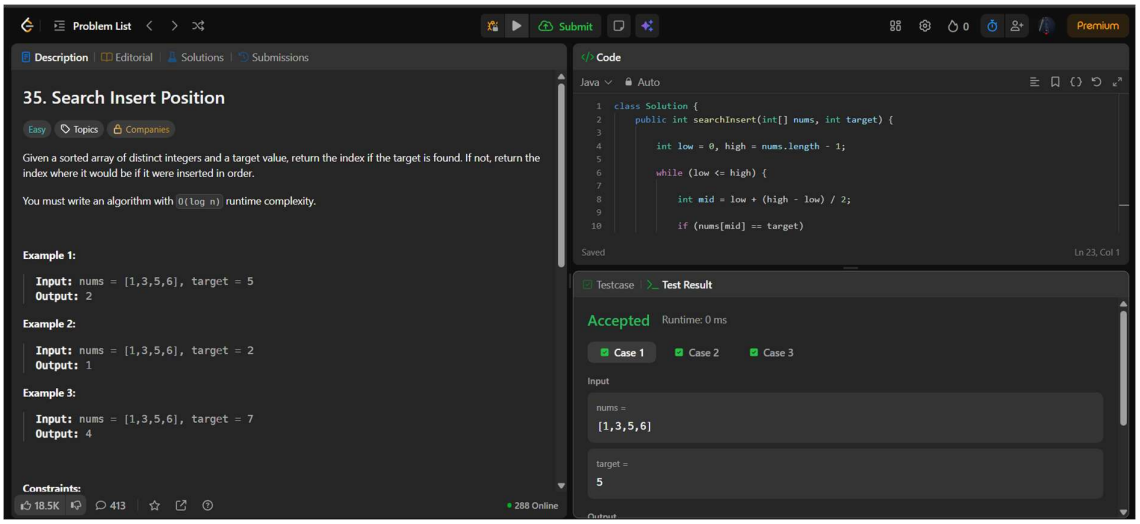
6.


Search...
Three 90 Ending
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Compilation Completed
Case 1
Input:
1 2 3 4 5
Your Output:
5 1 2 3 4
Expected Output:
5 1 2 3 4
Java (21)
Start Timer
class Solution {
public void rotate(int[] arr) {
int n = arr.length;
int last = arr[n - 1];
for (int i = n - 1; i > 0; i--) {
arr[i] = arr[i - 1];
}
arr[0] = last;
}
}
Custom Input
Compile & Run
Submit

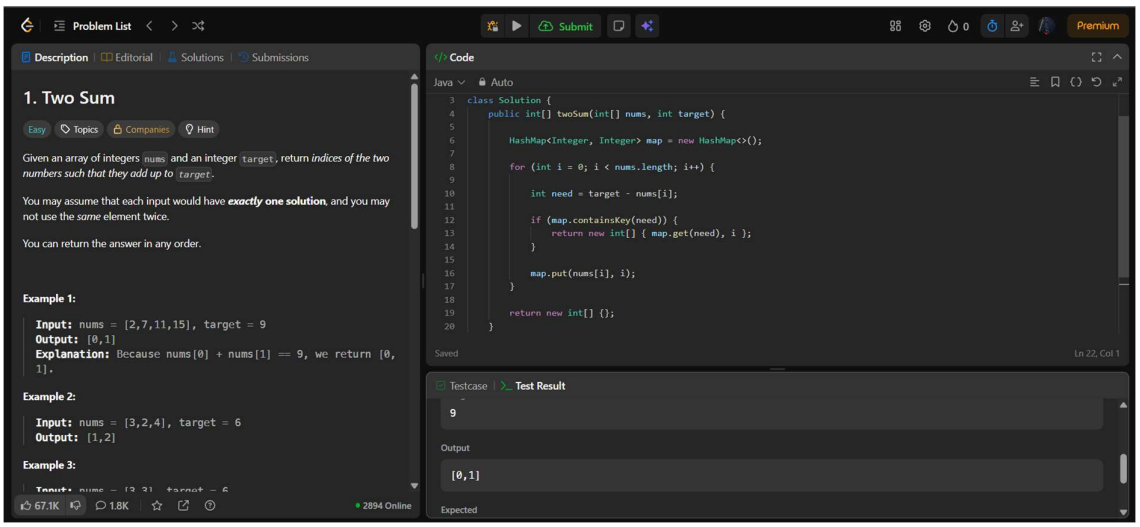
7.


Search...
Three 90 Ending
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Compilation Completed
Case 1
Input:
arr[] =
1 2 3 -2 5
Your Output:
9
Expected Output:
9
Java (21)
Start Timer
class Solution {
int maxSubarraySum(int[] arr) {
int currentSum = arr[0];
int maxSum = arr[0];
for (int i = 1; i < arr.length; i++) {
currentSum = Math.max(arr[i], currentSum + arr[i]);
maxSum = Math.max(maxSum, currentSum);
}
return maxSum;
}
}
Ctrl
Custom Input
Compile & Run
Submit

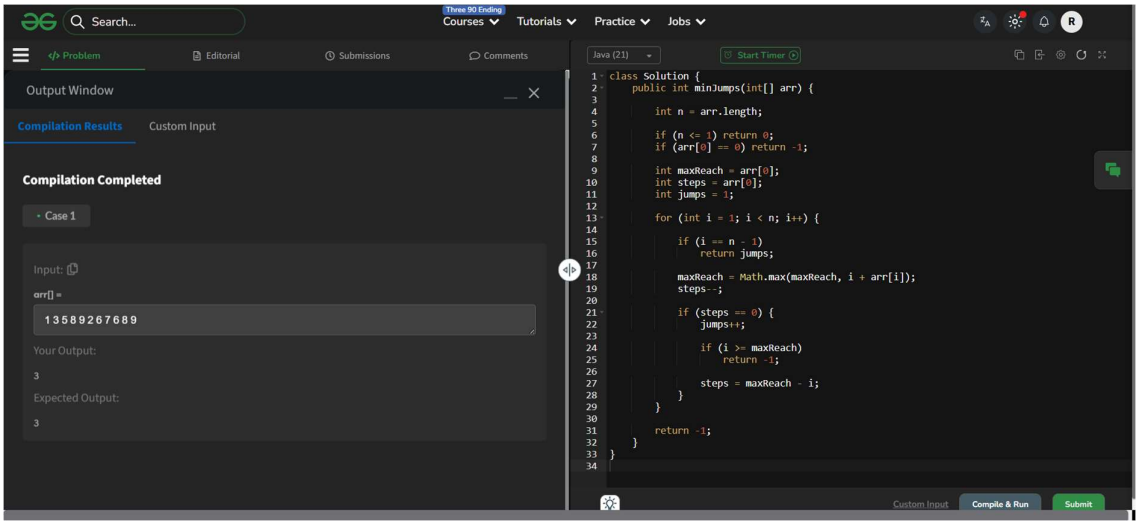
8.


Problem List
Submit
Premium
Description | Editorial | Solutions | Submissions
35. Search Insert Position
Easy Topics Companies
Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.
You must write an algorithm with O(log n) runtime complexity.
Example 1:
Input: nums = [1,3,5,6], target = 5
Output: 2
Example 2:
Input: nums = [1,3,5,6], target = 2
Output: 1
Example 3:
Input: nums = [1,3,5,6], target = 7
Output: 4
Constraints:
18.5K 413 288 Online
Code
Java Auto
class Solution {
public int searchInsert(int[] nums, int target) {
int low = 0, high = nums.length - 1;
while (low <= high) {
int mid = low + (high - low) / 2;
if (nums[mid] == target)
Saved Ln 23, Col 1
Testcase | Test Result
Accepted Runtime: 0 ms
Case 1 Case 2 Case 3
Input
nums =
[1,3,5,6]
target =
5

9.


Problem List
Submit
Premium
Description | Editorial | Solutions | Submissions
1. Two Sum
Easy Topics Companies Hint
Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.
You may assume that each input would have exactly one solution, and you may not use the same element twice.
You can return the answer in any order.
Example 1:
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
Example 2:
Input: nums = [3,2,4], target = 6
Output: [1,2]
Example 3:
67.1K 1.8K 2894 Online
Code
Java Auto
class Solution {
public int[] twoSum(int[] nums, int target) {
HashMap<Integer, Integer> map = new HashMap<>();
for (int i = 0; i < nums.length; i++) {
int need = target - nums[i];
if (map.containsKey(need)) {
return new int[] { map.get(need), i };
}
map.put(nums[i], i);
}
return new int[] {};
}
Saved Ln 22, Col 1
Testcase | Test Result
9
Output
[0,1]
Expected

10.


Search...
Three 90 Ending
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Compilation Completed
Case 1
Input:
arr[] =
1 3 5 8 9 2 6 7 6 8 9
Your Output:
3
Expected Output:
3
Java (21)
Start Timer
class Solution {
public int minJumps(int[] arr) {
int n = arr.length;
if (n <= 1) return 0;
if (arr[0] == 0) return -1;
int maxReach = arr[0];
int steps = arr[0];
int jumps = 1;
for (int i = 1; i < n; i++) {
if (i == n - 1)
return jumps;
maxReach = Math.max(maxReach, i + arr[i]);
steps--;
if (steps == 0) {
jumps++;
if (i >= maxReach)
return -1;
steps = maxReach - i;
}
}
return -1;
}
}
Custom Input
Compile & Run
Submit